ating Systems (CSD204): Graded Lab Assignment 2

Submitting this Assignment:

You will submit (upload) this assignment on Blackboard. Email submissions will not be accepted. Make sure that you upload the correct and complete file for the given assignment.

1. The following two files are required to be uploaded on Blackboard:
   - **One Java file** containing the complete code. (Multiple files are not allowed)
     - ✓ Project Name in Netbeans: **osgla2**
     - ✓ Class Name: **PageReplacement**
     - ✓ The java file must contain your name and roll no (as comments).
   - Pdf - Paste your Java code after the given question in this document. Also, paste the screenshots of input screen and output screen.
     Name this document as **GLA2_OS2019_John_Doe.pdf** in case your name is John Doe **[You are required to strictly follow the naming convention.]**

2. Assignment submitted after due date and time will not be evaluated and a score of zero will be awarded for this assignment.

3. Materials copied from the Internet or otherwise will attract penalty as per the course policy.

Grading: This assignment has 10 points (with **weightage of 10%** in your overall 100 points).

Due Date: **11:30 pm, April 27, 2019**

Question

1. Write a GUI based Java program (sample template discussed in class) to implement the following Page Replacement algorithms and compare their performance:
   a. First In First Out (FIFO)
   b. Optimal Page Replacement
   c. Least Recently Used (LRU)

Calculate number of page faults by taking different number of frames, say, 3,4,5,6 and 7 frames.

Java Code

//paste here

```
//Made by Kabir Kanha Arora
//1710110165
//Shiv Nadar University
//CSE-2021
//15-Apr-2019

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.FocusEvent;
import java.awt.event.FocusListener;

public class PageReplacement extends JFrame {


    //Main JFrame
    private static JFrame mainframe;

    //Creating string with options
    private String[] arr = {"3", "4", "5", "6", "7"};
```

```
    //Creating combo box for choice
    private JComboBox<String> jcb_choice = new JComboBox<>(arr);

    private JLabel jlb_pf_fifo = new JLabel();
    private JLabel jlb_pf_lru = new JLabel();
    private JLabel jlb_pf_opt = new JLabel();

    //Creating text fields
    private JTextField jtf_refstr = new JTextField("Enter comma-separated reference string");

    //Keeps track of the number of frames
    private static int cnt_frame;

    //Keeps track of the number of elements in the reference string
    private static int cnt_refstr;

    //ArrayLists
    private static int[] arr_frames;

    //JPanels
    private JPanel panel_fifo;
    private JPanel panel_lru;
    private JPanel panel_opt;

    //Page Faults
    private int pf_fifo;
    private int pf_lru;
    private int pf_opt;

    private DefaultTableModel model_fifo = new DefaultTableModel();
    private DefaultTableModel model_lru = new DefaultTableModel();
    private DefaultTableModel model_opt = new DefaultTableModel();

    //Initialising JTables
    private JTable table_FIFO = new JTable(model_fifo);
    private JTable table_LRU = new JTable(model_lru);
    private JTable table_OPT = new JTable(model_opt);

    private static String[] sarr_frames;

    private PageReplacement() {

        jtf_refstr.addFocusListener(new FocusListener() {
            @Override
            public void focusGained(FocusEvent e) {
                if (jtf_refstr.getText().equals("Enter comma-separated reference string"))
                    jtf_refstr.setText("");
            }

            @Override
            public void focusLost(FocusEvent e) {
                if (jtf_refstr.getText().equals(""))
                    jtf_refstr.setText("Enter comma-separated reference string");
            }
        });

        //Creating a JButton
        JButton jbt_compute = new JButton("COMPUTE");
        jbt_compute.addActionListener(e -> {
            model_fifo.setRowCount(0);
            model_fifo.setColumnCount(0);
            model_lru.setRowCount(0);
            model_lru.setColumnCount(0);
            model_opt.setRowCount(0);
            model_opt.setColumnCount(0);

            //Getting number of frames
            cnt_frame = jcb_choice.getSelectedIndex() + 3;

            //Flag for checking if any conditions have been violated
            int flag = 0;

            //Getting burst times into an array
            if (!(jtf_refstr.getText().equals("") || jtf_refstr.getText().equals("Enter comma-separated
reference string"))) {
                sarr_frames = jtf_refstr.getText().replaceAll("\\s", "").split(",", -1);
                cnt_refstr = sarr_frames.length;
                arr_frames = new int[cnt_refstr];
                for (int i = 0; i < cnt_refstr && flag == 0; ++i) {
                    try {
                        arr_frames[i] = Integer.parseInt(sarr_frames[i]);
                    } catch (Exception ex) {
                        JOptionPane.showMessageDialog(null, "Invalid values in reference string.");
                        flag = -1;
```

```
                }
            }
        } else {
            JOptionPane.showMessageDialog(null, "Please enter a reference string.");
            flag = -1;
        }
        //Resize frame to accommodate tables
        if (flag == 0) {
            panel_fifo.setVisible(true);
            panel_lru.setVisible(true);
            panel_opt.setVisible(true);
            mainframe.setSize(575, 650);
            Object[][] matrix_fifo = new Object[cnt_frame][cnt_refstr];
            Queue_FIFO q1 = new Queue_FIFO(matrix_fifo, cnt_frame);
            for (int i = 0; i < cnt_refstr; ++i) {
                q1.enqueue(arr_frames[i]);
                q1.show();
            }
            for (int i = 0; i < cnt_refstr; ++i) {
                model_fifo.addColumn(sarr_frames[i]);
            }
            for (int i = 0; i < cnt_frame; ++i) {
                model_fifo.addRow(matrix_fifo[i]);
            }
            table_FIFO.getTableHeader().setReorderingAllowed(false);
            table_FIFO.getTableHeader().setResizingAllowed(false);
            table_FIFO.setEnabled(false);
            table_FIFO.setGridColor(Color.RED);
            pf_fifo = q1.getFifo_pf_counter();
            jlb_pf_fifo.setText("Page Faults: " + pf_fifo);

            Object[][] matrix_lru = new Object[cnt_frame][cnt_refstr];
            Queue_LRU q2 = new Queue_LRU(matrix_lru, cnt_frame);
            for (int i = 0; i < cnt_refstr; ++i) {
                q2.enqueue(arr_frames[i]);
                q2.show();
            }
            for (int i = 0; i < cnt_refstr; ++i) {
                model_lru.addColumn(sarr_frames[i]);
            }
            for (int i = 0; i < cnt_frame; ++i) {
                model_lru.addRow(matrix_lru[i]);
            }
            table_LRU.getTableHeader().setReorderingAllowed(false);
            table_LRU.getTableHeader().setResizingAllowed(false);
            table_LRU.setEnabled(false);
            table_LRU.setGridColor(Color.GREEN);
            pf_lru = q2.getLru_pf_counter();
            jlb_pf_lru.setText("Page Faults: " + pf_lru);

            Object[][] matrix_opt = new Object[cnt_frame][cnt_refstr];
            Queue_OPT q3 = new Queue_OPT(matrix_opt, cnt_frame, arr_frames);
            for (int i = 0; i < cnt_refstr; ++i) {
                q3.enqueue(arr_frames[i], i);
                q3.show();
            }
            for (int i = 0; i < cnt_refstr; ++i) {
                model_opt.addColumn(sarr_frames[i]);
            }
            for (int i = 0; i < cnt_frame; ++i) {
                model_opt.addRow(matrix_opt[i]);
            }
            table_OPT.getTableHeader().setReorderingAllowed(false);
            table_OPT.getTableHeader().setResizingAllowed(false);
            table_OPT.setEnabled(false);
            table_OPT.setGridColor(Color.BLUE);
            pf_opt = q3.getOpt_pf_counter();
            jlb_pf_opt.setText("Page Faults: " + pf_opt);
        }
    });

    //Creating JPanels for various purposes
    JPanel panel_main = new JPanel();
    panel_main.setLayout(new BoxLayout(panel_main, BoxLayout.PAGE_AXIS));
    JPanel panel_maintag = new JPanel();
    panel_maintag.setLayout(new BoxLayout(panel_maintag, BoxLayout.LINE_AXIS));
    JPanel panel_frames = new JPanel();
    panel_frames.setLayout(new BoxLayout(panel_frames, BoxLayout.LINE_AXIS));
    JPanel panel_refstr = new JPanel();
    panel_refstr.setLayout(new BoxLayout(panel_refstr, BoxLayout.LINE_AXIS));
    JPanel panel_button = new JPanel();
    panel_button.setLayout(new BoxLayout(panel_button, BoxLayout.LINE_AXIS));
    panel_fifo = new JPanel();
    panel_fifo.setLayout(new BoxLayout(panel_fifo, BoxLayout.PAGE_AXIS));
```

```
        panel_lru = new JPanel();
        panel_lru.setLayout(new BoxLayout(panel_lru, BoxLayout.PAGE_AXIS));
        panel_opt = new JPanel();
        panel_opt.setLayout(new BoxLayout(panel_opt, BoxLayout.PAGE_AXIS));

        //Setting the correct sizes
        //Creating JLabels
        JLabel jlb_frames = new JLabel("Number of Frames");
        jlb_frames.setPreferredSize(new Dimension(200, 20));
        jlb_frames.setMaximumSize(new Dimension(200, 20));
        jcb_choice.setPreferredSize(new Dimension(300, 20));
        jcb_choice.setMaximumSize(new Dimension(300, 20));
        JLabel jlb_refstr = new JLabel("Reference String");
        jlb_refstr.setPreferredSize(new Dimension(200, 20));
        jlb_refstr.setMaximumSize(new Dimension(200, 20));
        jtf_refstr.setPreferredSize(new Dimension(300, 20));
        jtf_refstr.setMaximumSize(new Dimension(300, 20));

        //Adding panels to their respective locations
        JLabel jlb_maintag = new JLabel("Page Replacement Algorithms");
        panel_maintag.add(jlb_maintag);

        panel_frames.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_frames.add(jlb_frames);
        panel_frames.add(jcb_choice);
        panel_frames.add(Box.createRigidArea(new Dimension(30, 0)));

        panel_refstr.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_refstr.add(jlb_refstr);
        panel_refstr.add(jtf_refstr);
        panel_refstr.add(Box.createRigidArea(new Dimension(30, 0)));

        panel_button.add(jbt_compute);

        JLabel jlb_fifo = new JLabel("First In, First Out (FIFO)");
        jlb_fifo.setFont(new Font("Arial", Font.BOLD, 16));
        jlb_pf_fifo.setFont(new Font("Arial", Font.ITALIC, 12));
        JLabel jlb_lru = new JLabel("Least Recently Used (LRU)");
        jlb_lru.setFont(new Font("Arial", Font.BOLD, 16));
        jlb_pf_lru.setFont(new Font("Arial", Font.ITALIC, 12));
        JLabel jlb_opt = new JLabel("Optimal Page Replacement");
        jlb_opt.setFont(new Font("Arial", Font.BOLD, 16));
        jlb_pf_opt.setFont(new Font("Arial", Font.ITALIC, 12));

        panel_fifo.add(jlb_fifo);
        panel_fifo.add(jlb_pf_fifo);
        //JScrollPanes
        JScrollPane sp_fifo = new JScrollPane(table_FIFO);
        panel_fifo.add(sp_fifo);

        panel_lru.add(jlb_lru);
        panel_lru.add(jlb_pf_lru);
        JScrollPane sp_lru = new JScrollPane(table_LRU);
        panel_lru.add(sp_lru);

        panel_opt.add(jlb_opt);
        panel_opt.add(jlb_pf_opt);
        JScrollPane sp_opt = new JScrollPane(table_OPT);
        panel_opt.add(sp_opt);

        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_maintag);
        panel_main.add(Box.createRigidArea(new Dimension(0, 20)));
        panel_main.add(panel_frames);
        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_refstr);
        panel_main.add(Box.createRigidArea(new Dimension(0, 20)));
        panel_main.add(jbt_compute);
        panel_main.add(Box.createRigidArea(new Dimension(0, 25)));
        panel_main.add(panel_fifo);
        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_lru);
        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_opt);
        panel_fifo.setVisible(false);
        panel_lru.setVisible(false);
        panel_opt.setVisible(false);

        this.add(panel_main);

        //Setting fonts
        jtf_refstr.setFont(new Font("Arial", Font.ITALIC, 11));
        jlb_maintag.setFont(new Font("Times New Roman", Font.PLAIN, 20));
```

```
        //Setting alignments
        jbt_compute.setAlignmentX(CENTER_ALIGNMENT);
    }

    public static void main(String[] args) {
        mainframe = new PageReplacement();
        mainframe.setSize(500, 250);
        mainframe.setLocation(500, 20);
        mainframe.setResizable(false);
        mainframe.setTitle("OS GLA 2");
        mainframe.pack();
        mainframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        mainframe.setVisible(true);
    }
}

class Queue_FIFO {
    private int size = 0;
    private int max_size;
    private int cnt = 0;
    private Object[][] matrix;
    private Object[] queue;
    private int[] counter;
    private int flag;
    private int fifo_pf_counter = 0;

    Queue_FIFO(Object[][] matrix, int frames) {
        max_size = frames;
        queue = new Object[max_size];
        counter = new int[max_size];
        this.matrix = matrix;
        flag = 0;
    }

    void enqueue(Object data) {
        for (int i = 0; i < size; ++i) {
            if (queue[i] == data) {
                flag = 1;
            }
        }
        if (flag == 0) {
            fifo_pf_counter++;
            if (size < max_size) {
                queue[size] = data;
                counter[size] = max_size;
                for (int i = 0; i < max_size; ++i) {
                    if (i != size) {
                        counter[i]--;
                    }
                }
                ++size;
            } else {
                int min = Integer.MAX_VALUE;
                int min_val = Integer.MAX_VALUE;
                for (int i = 0; i < max_size; ++i) {
                    if (counter[i] < min_val) {
                        min_val = counter[i];
                        min = i;
                    }
                }
                queue[min] = data;
                counter[min] = max_size;
                for (int i = 0; i < max_size; ++i) {
                    if (i != min) {
                        counter[i]--;
                    }
                }
            }
        }
        flag = 0;
    }

    void show() {
        for (int i = 0; i < max_size; ++i) {
            matrix[i][cnt] = queue[i];
        }
        cnt++;
    }

    int getFifo_pf_counter() {
        return fifo_pf_counter;
    }
}
```

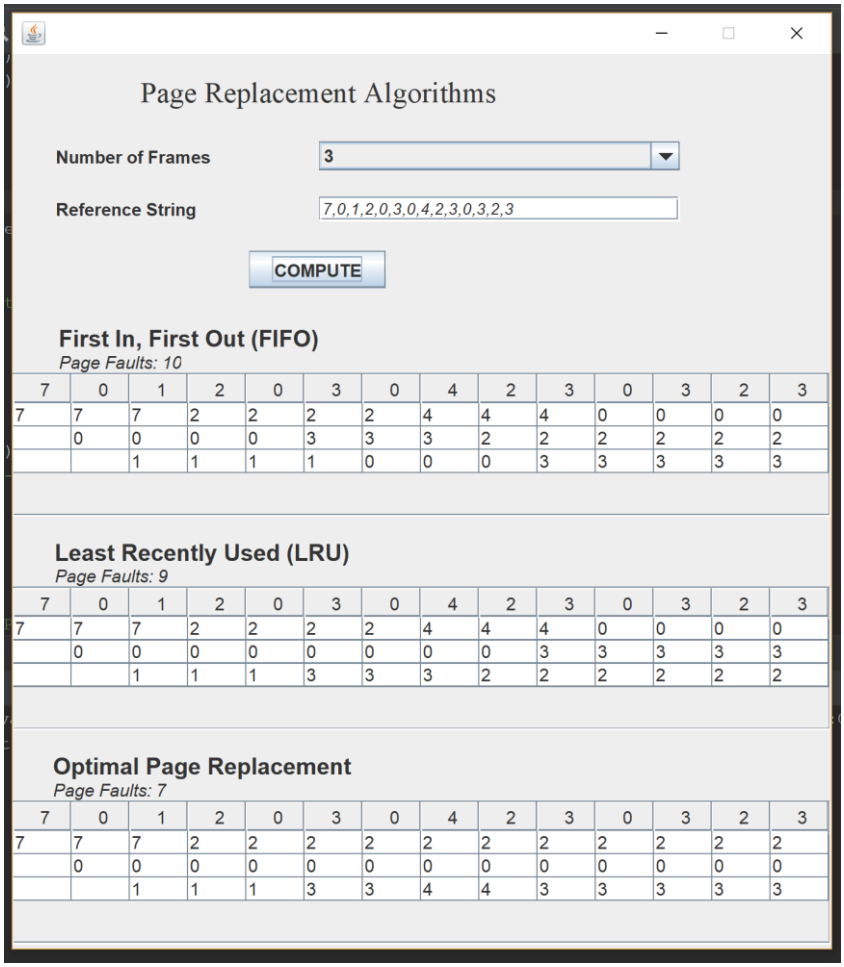
```
class Queue_LRU {
    private int size = 0;
    private int max_size;
    private int cnt = 0;
    private Object[][] matrix;
    private Object[] queue;
    private int[] counter;
    private int flag;
    private int lru_pf_counter = 0;

    Queue_LRU(Object[][] matrix, int frames) {
        max_size = frames;
        queue = new Object[max_size];
        counter = new int[max_size];
        this.matrix = matrix;
        flag = 0;
    }

    void enqueue(Object data) {
        for (int i = 0; i < size; ++i) {
            if (queue[i] == data) {
                flag = 1;
                counter[i] = max_size;
                for (int j = 0; j < max_size; ++j) {
                    if (j != i) {
                        counter[j]--;
                    }
                }
            }
        }
        if (flag == 0) {
            lru_pf_counter++;
            if (size < max_size) {
                queue[size] = data;
                counter[size] = max_size;
                for (int i = 0; i < max_size; ++i) {
                    if (i != size) {
                        counter[i]--;
                    }
                }
                ++size;
            } else {
                int min = Integer.MAX_VALUE;
                int min_val = Integer.MAX_VALUE;
                for (int i = 0; i < max_size; ++i) {
                    if (counter[i] < min_val) {
                        min_val = counter[i];
                        min = i;
                    }
                }
                queue[min] = data;
                counter[min] = max_size;
                for (int i = 0; i < max_size; ++i) {
                    if (i != min) {
                        counter[i]--;
                    }
                }
            }
        }
        flag = 0;
    }

    void show() {
        for (int i = 0; i < max_size; ++i) {
            matrix[i][cnt] = queue[i];
        }
        cnt++;
    }

    int getLru_pf_counter() {
        return lru_pf_counter;
    }
}

class Queue_OPT {
    private int size = 0;
    private int max_size;
    private int cnt = 0;
    private Object[][] matrix;
    private Object[] queue;
    private int[] counter;
    private int flag;
    private int opt_pf_counter = 0;
    private Object[] refArray = new Object[100];
```

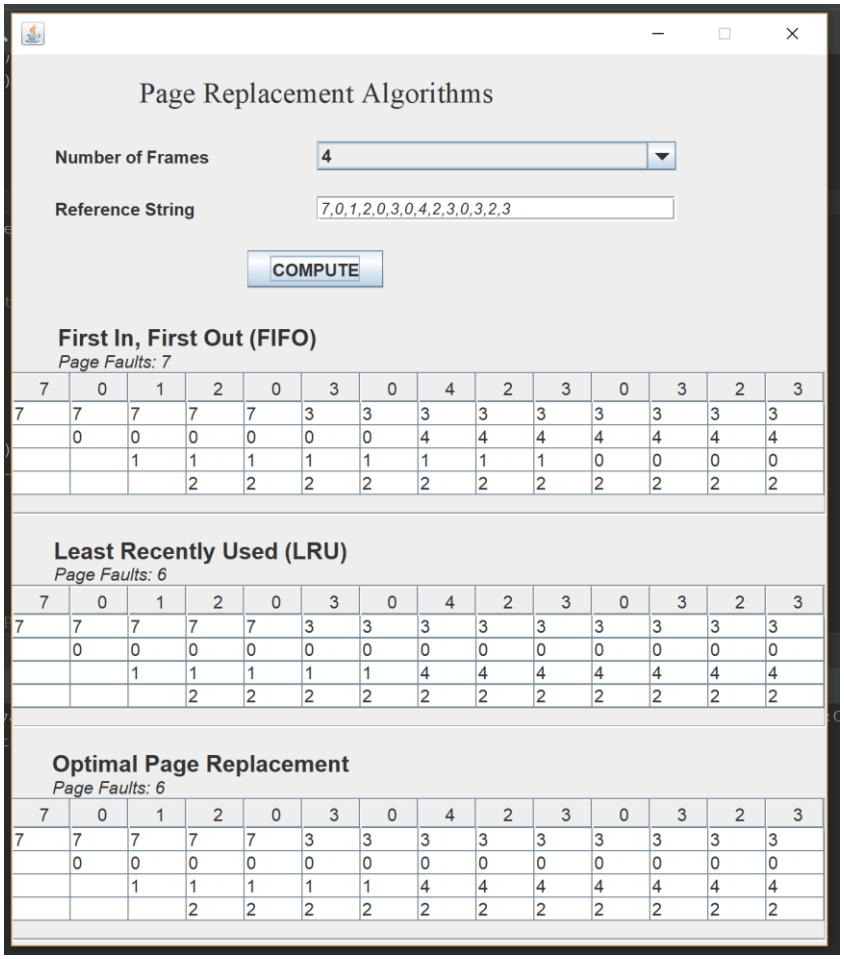
```
    Queue_OPT(Object[][] matrix, int frames, int[] refArray) {
        max_size = frames;
        queue = new Object[max_size];
        counter = new int[max_size];
        this.matrix = matrix;
        flag = 0;
        for (int i = 0; i < max_size; ++i) {
            counter[i] = Integer.MAX_VALUE;
        }
        for (int i = 0; i < refArray.length; ++i) {
            this.refArray[i] = refArray[i];
        }
    }

    void enqueue(Object data, int pos) {
        for (int i = 0; i < size; ++i) {
            if (queue[i] == data) {
                flag = 1;
            }
        }
        if (flag == 0) {
            opt_pf_counter++;
            if (size < max_size) {
                queue[size] = data;
                ++size;
            } else {
                //Replacement
                for (int i = 0; i < max_size; ++i) {
                    counter[i] = Integer.MAX_VALUE;
                }
                for (int i = 0; i < max_size; ++i) {
                    for (int j = pos + 1; j < refArray.length; ++j) {
                        if (queue[i] == refArray[j]) {
                            counter[i] = j - (pos);
                        }
                    }
                }
                int max = Integer.MIN_VALUE;
                int max_val = Integer.MIN_VALUE;
                for (int i = 0; i < max_size; ++i) {
                    if (counter[i] > max_val) {
                        max_val = counter[i];
                        max = i;
                    }
                }
                queue[max] = data;
            }
        }
        flag = 0;
    }

    void show() {
        for (int i = 0; i < max_size; ++i) {
            matrix[i][cnt] = queue[i];
        }
        cnt++;
    }

    int getOpt_pf_counter() {
        return opt_pf_counter;
    }
}
```

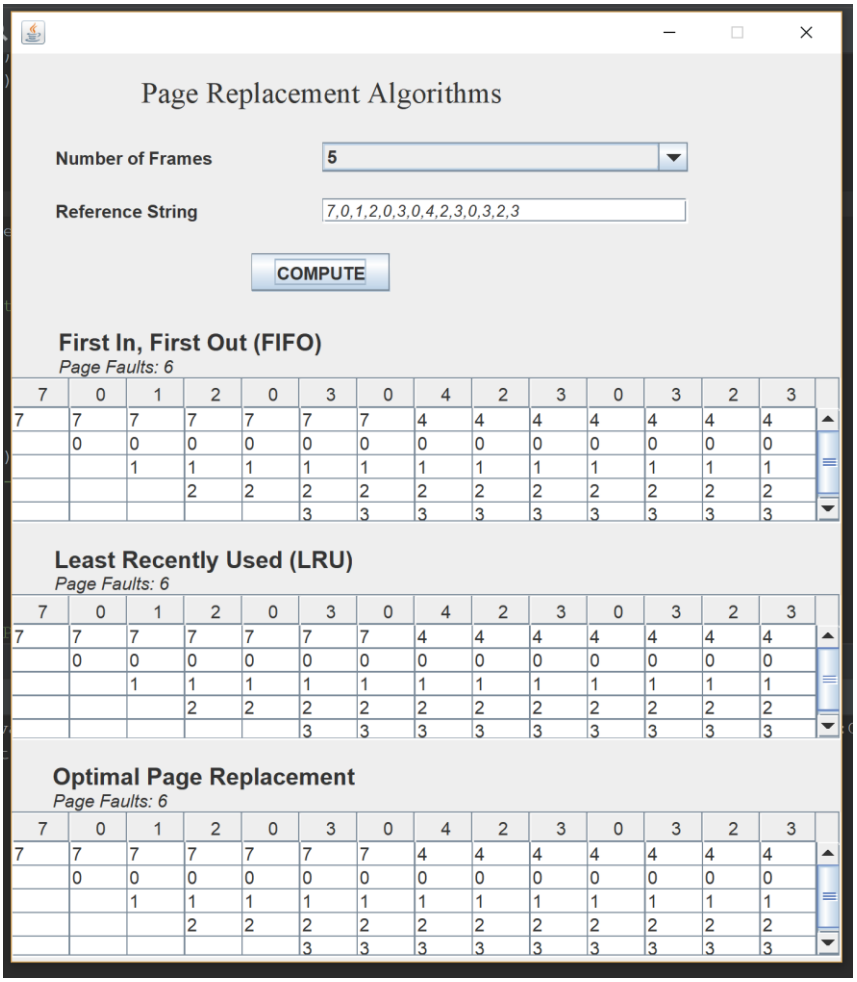
Screenshots

5 screenshots, one each from Frames 3, 4, 5, 6, 7, preferably on same reference string

//paste here


Page Replacement Algorithms
Number of Frames
3
Reference String
7,0,1,2,0,3,0,4,2,3,0,3,2,3
COMPUTE
First In, First Out (FIFO)
Page Faults: 10
Least Recently Used (LRU)
Page Faults: 9
Optimal Page Replacement
Page Faults: 7

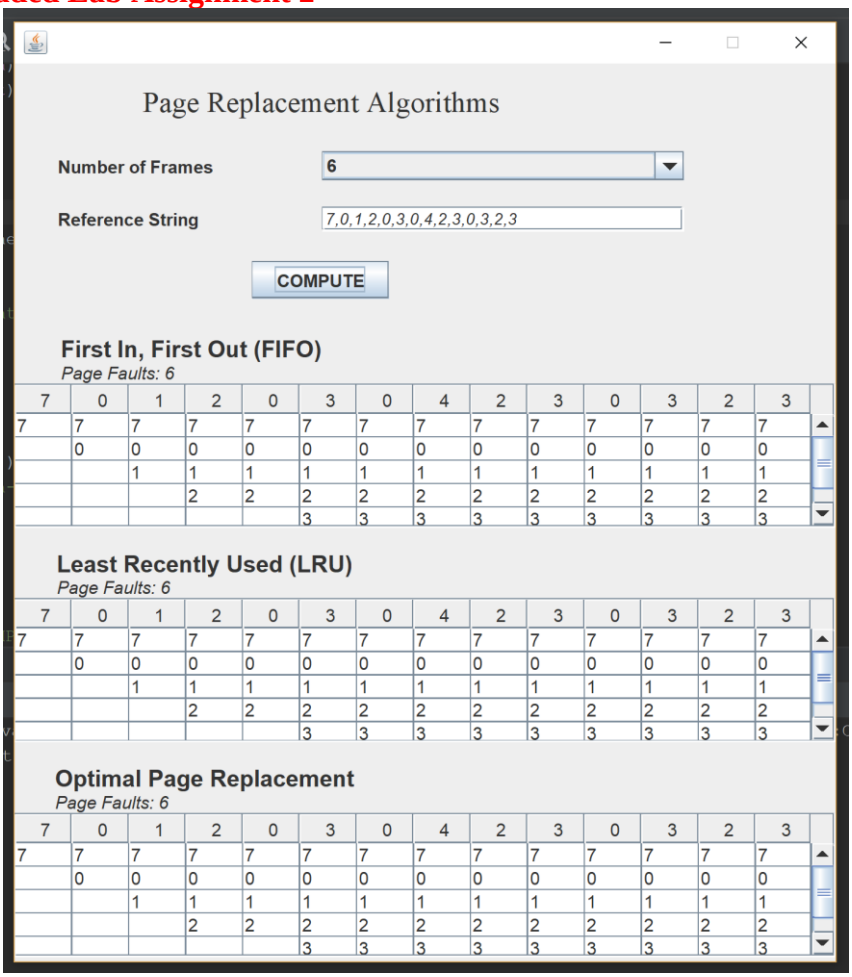

Page Replacement Algorithms
Number of Frames
4
Reference String
7,0,1,2,0,3,0,4,2,3,0,3,2,3
COMPUTE
First In, First Out (FIFO)
Page Faults: 7
Least Recently Used (LRU)
Page Faults: 6
Optimal Page Replacement
Page Faults: 6

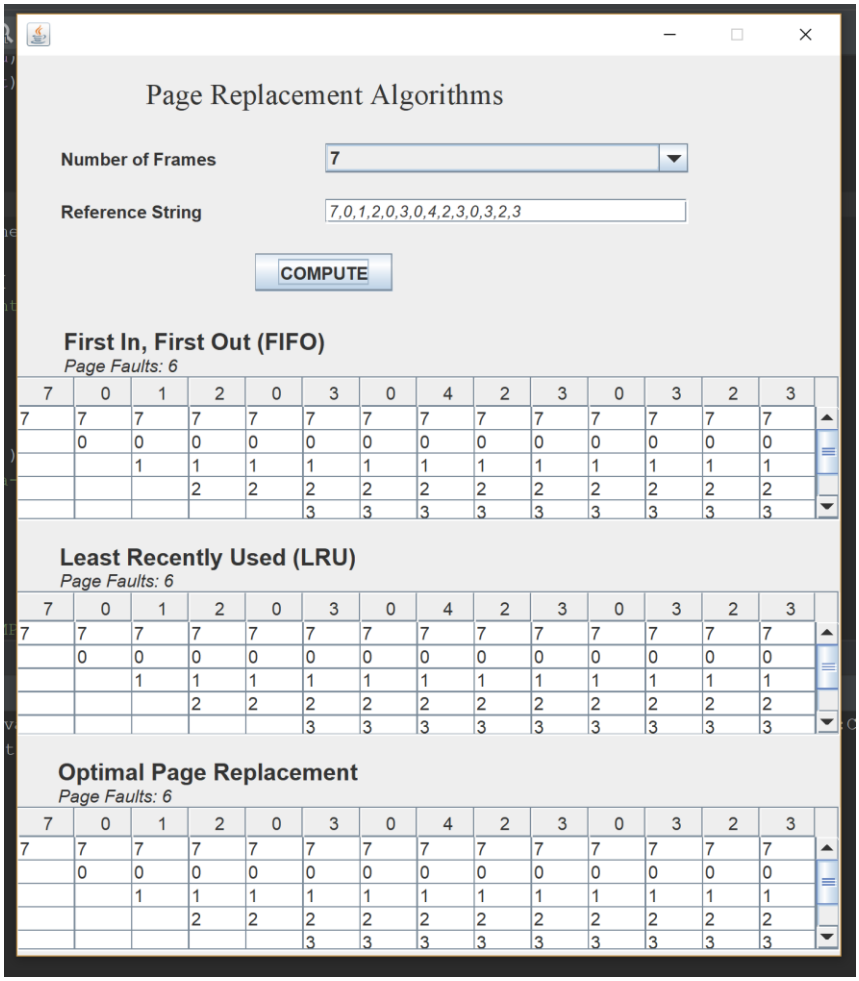


7	0	1	2	0	3	0	4	2	3	0	3	2	3
7	7	7	7	7	7	7	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2
					3	3	3	3	3	3	3	3	3

7	0	1	2	0	3	0	4	2	3	0	3	2	3
7	7	7	7	7	7	7	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2
					3	3	3	3	3	3	3	3	3

7	0	1	2	0	3	0	4	2	3	0	3	2	3
7	7	7	7	7	7	7	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2
					3	3	3	3	3	3	3	3	3


Page Replacement Algorithms
Number of Frames
6
Reference String
7,0,1,2,0,3,0,4,2,3,0,3,2,3
COMPUTE
First In, First Out (FIFO)
Page Faults: 6
Least Recently Used (LRU)
Page Faults: 6
Optimal Page Replacement
Page Faults: 6


Page Replacement Algorithms
Number of Frames
7
Reference String
7,0,1,2,0,3,0,4,2,3,0,3,2,3
COMPUTE
First In, First Out (FIFO)
Page Faults: 6
Least Recently Used (LRU)
Page Faults: 6
Optimal Page Replacement
Page Faults: 6